

# Guns 2 Ammo Business System

## Client Needs Analysis and Master Stabilization Plan

### Executive conclusion

Guns 2 Ammo does not primarily need another website redesign or more isolated plugins.

The business needs one dependable operating system that connects:

- Membership billing and cancellation
- Customer accounts and linked family members
- Waivers and range eligibility
- Lane and class reservations
- Front-counter check-in
- POS and retail inventory
- Lipsey's product data
- Email, SMS, chatbot, and marketing contacts
- Website content management
- Staff permissions, instructions, and reporting

The repository already contains most of these systems. WordPress is intentionally used as the central source of truth for bookings, memberships, orders, waivers, payments, POS sales, and customer information. The custom dashboard and other services operate as clients of WordPress rather than separate databases.

However, the current problem is:

**The repository, production site, data, staff workflow, and client expectations are not yet synchronized.**

A feature existing in GitHub does not mean it has been deployed, configured, tested with live data, explained to employees, or accepted by the client.

---

# 1. What the client actually needs

## 1.1 Reliable membership cancellation

### Client's expectation

When staff cancels a membership:

1. The local membership must become cancelled.
2. Future recurring billing must stop.
3. The remote payment subscription must be cancelled.
4. Staff must receive a clear success or failure result.
5. The cancellation must appear in an audit log.

### Current repository status

The latest Memberistic source contains Stripe subscription cancellation logic and hooks it to local membership status changes.

The July 13 unified release also states that Memberistic 1.12.0 includes Stripe cancellation propagation.

### Remaining risk

The current flow changes the local membership status first and then attempts to cancel Stripe. When Stripe fails, the system records an error, but the local membership may already display as cancelled while billing remains active.

That is not sufficient for a financial operation.

### Required solution

Replace the simple status hook with a controlled cancellation workflow:

1. Staff clicks **Cancel Membership**.
2. System loads the remote subscription.
3. System sends the cancellation request to the payment gateway.
4. On confirmed success:
5. Mark local membership cancelled.
6. Store gateway cancellation ID and timestamp.
7. Send customer confirmation.
8. Add an audit-log entry.
9. On failure:

10. Keep the membership in `cancel_failed` or `cancel_pending` .

11. Show the actual error to authorized staff.

12. Create an operations alert.

13. Retry through a reconciliation job.

A nightly reconciliation should compare all locally active/cancelled recurring memberships against the payment processor.

---

## 1.2 Correct address and business information in every email

### Client's expectation

All booking, membership, receipt, waiver, chatbot, and contact emails must show the correct:

- Business name
- Address
- Phone
- Email
- Hours

### Current repository status

The repository found the original cause: the booking plugin had its own incomplete address value, `"Mesa, Arizona"`, separate from the theme's central business-information settings.

The newer code added fallback helpers so booking emails, invoices, receipts, payment pages, AI replies, and settings read the shared business information.

### Required solution

This needs a production verification pass:

- Deploy the correct plugin release.
- Remove or correct the old `g2ab_business_address` option.
- Confirm the Customizer contains the full address.
- Test all booking lifecycle emails.
- Test membership emails.
- Test Formistic replies and newsletter emails.
- Test receipts and PDF invoices.
- Save screenshots and delivered-message evidence.

This issue should not be marked completed until actual emails arrive with the correct address.

---

## 1.3 Backend calendar for classes and bookings

### Client's expectation

Staff needs a visual calendar showing:

- Lane reservations
- Classes and courses
- Available capacity
- Customer names
- Payment state
- Check-in state
- Rescheduling and cancellation actions

### Current repository status

A WordPress admin FullCalendar interface already exists. It registers a **Calendar** submenu and requires the `manage_g2ab_bookings` capability.

It includes:

- Day, week, and month views
- Resource and status filters
- Drag-to-reschedule
- Booking details
- Cancellation and quick actions

The calendar also uses a locally bundled FullCalendar asset, with a CDN fallback, to avoid security plugins blocking it.

### Diagnosis

The client's missing calendar is likely one of these:

- Production is running an older Booking Engine version.
- The calendar class is not booting.
- The employee role lacks `manage_g2ab_bookings`.
- A stale release ZIP was deployed.
- The submenu is hidden by another admin-menu modification.
- JavaScript or REST requests are failing.

### Required solution

Verify these in order:

1. Confirm the executing production version.
2. Confirm `G2AB_Admin_Calendar::instance()` is loaded.

3. Confirm the client's role has `manage_g2ab_bookings`.
  4. Open the calendar while monitoring browser console and REST requests.
  5. Test lane bookings and class registrations.
  6. Add a direct **Classes Calendar** shortcut for staff if the combined calendar is too complex.
- 

## 1.4 Website corrections and control handoff

### Client's expectation

The client wants:

- Small page corrections completed.
- Ability to update content without asking the developer.
- Clear knowledge of which settings control each page.
- Appropriate access without exposing sensitive technical settings.

### Current repository status

The theme and G2A Theme Control already expose page content, images, CTA links, contact information, hours, inventory, and other content through WordPress editing screens and Customizer settings.

### Actual missing deliverable

The missing piece is **handoff**, not necessarily code.

Create:

- A page-by-page correction register.
- An approved-copy column.
- A production-completed column.
- A client-accepted column.
- A restricted `G2A Content Manager` role.
- A short website editing guide.
- A screen-recorded content editing walkthrough.

Do not give the client unrestricted administrator access for normal page editing.

---

## 1.5 Lipsey's products, categories, and images

### Client's expectation

Lipsey's integration must reliably import:

- Firearms

- Accessories
- Categories
- Stock
- Pricing
- Product details
- Images

## **Current repository status**

The system supports a Lipsey's catalog API and stores normalized product fields, including item type, category, price, stock, FFL requirement, and image filename.

The POS API includes:

- Credential testing
- API catalog synchronization
- CSV import
- Inventory synchronization
- Category management

A previous audit found a major dual-account risk: code resolving Lipsey's only by provider could select the lowest-ID account, making the second account unreachable. Blank account numbers could also cause one account to overwrite another.

## **Required solution**

Treat this as a structured data-integration project:

1. Create two clearly identified wholesaler connections:
2. Lipsey's Firearms
3. Lipsey's Accessories
4. Require unique, non-empty account numbers.
5. Test credentials independently.
6. Save a sanitized sample of each account's API response.
7. Compare returned fields between the two feeds.
8. Build a category mapping table:
9. Lipsey's type
10. Lipsey's group
11. Internal POS category
12. WooCommerce category
13. Validate image filenames and generated image URLs.
14. Mirror approved images locally instead of depending permanently on remote URLs.
15. Import first into the staging catalog.

16. Promote selected products to WooCommerce only after validation.
17. Never delete or zero WooCommerce products because an upstream feed temporarily fails.
18. Add sync logs showing:
19. Products received
20. Products inserted
21. Products updated
22. Products rejected
23. Missing images
24. Unknown categories

The system already preserves last-known-good storefront data when Lipsey's fails.

---

## 1.6 Otter Text, chatbot, and waiver retirement

These are two separate migrations:

### **Otter Text chatbot/SMS retirement**

The repository documents that Otter Text is an external embed and can be removed by deactivating its plugin, disabling its webhook, revoking its key, and cancelling the external service.

Before cancelling it, verify the replacement chatbot:

- Loads on desktop and mobile
- Answers current business questions correctly
- Captures customer details
- Sends contacts to the correct inbox
- Provides human escalation
- Stores conversation records
- Handles downtime gracefully
- Has analytics and error logs

Only cancel Otter Text after this acceptance checklist passes.

### **Otter Waiver retirement**

Do not cancel Otter Waiver before exporting and mirroring its PDFs.

The importer can:

- Import waiver contacts
- Deduplicate repeat signings
- Match users by email

- Mark waiver status
- Mirror PDFs into protected local storage

The documented dataset contained 1,922 rows and 1,793 unique people.

Required sequence:

1. Export the newest complete Otter Waiver CSV.
2. Test-import a small sample.
3. Verify names, emails, signed dates, minors, and PDFs.
4. Run the full WP-CLI import with PDF mirroring.
5. Compare source row count, unique-person count, imported count, and failed count.
6. Match imported waivers against Memberistic people and WordPress users.
7. Produce an unmatched-waiver report.
8. Confirm imported waivers satisfy the range check-in process.
9. Preserve the source export and import report.
10. Cancel Otter Waiver only after verification.

---

## 1.7 Customer login problems, especially for older customers

The codebase previously found that Cloudflare caching could cause a successful login to redirect customers to a cached logged-out account page.

Code fixes alone will not solve all usability problems.

### Required customer-access improvements

- Keep `/login/`, `/account/`, and `/my-account/` completely outside page caching.
- Add a prominent **Forgot Password** flow.
- Add one-click password setup links for newly imported members.
- Consider passwordless email login for older customers.
- Use larger form controls and simpler error messages.
- Avoid requiring customers to understand whether they are a “member,” “WordPress user,” or “WooCommerce customer.”
- Give counter staff a safe **Send Login Link** action.
- Record login failures by reason without logging passwords.

Memberistic intentionally separates member records from WordPress login accounts, which explains why some imported or walk-in members may not have working logins.

A reconciliation report should identify:

- Member with no WordPress user
- WordPress user with no Memberistic person
- Duplicate email accounts
- Invalid or missing email



- Linked profile with no primary membership
  - Expired membership still carrying an active role
- 

## **1.8 Front-counter workflow and employee training**

### **Client's expectation**

Employees need a simple answer to:

“What do I do when a customer walks into the range?”

### **Current repository status**

The repository already documents a full workflow covering:

- Opening the staff console
- Reviewing lanes and bookings
- Walk-in check-in
- QR self-check-in
- Waiver verification
- Lane assignment
- Payment status
- No-shows
- End-of-day process

### **Actual missing deliverable**

Employees do not need a 300-line technical document.

They need:

1. A one-page counter checklist.
2. A visual decision tree.
3. A test employee account.
4. A practice environment.
5. A short training session.
6. A manager escalation procedure.
7. A list of actions employees must never perform.

### **Counter workflow**

1. Search customer by email, phone, name, or QR.
2. Confirm identity.
3. Confirm membership status.
4. Confirm waiver status and expiration.
5. Confirm booking or create a walk-in.

6. Assign lane.
  7. Confirm payment.
  8. Complete check-in.
  9. Print or email receipt.
  10. Record exceptions in staff notes.
- 

## 1.9 Contact cleanup and duplicate management

Formistic is now the intended contact-form, inbox, and newsletter system. It includes an idempotent migration from the older contact plugin.

However, preventing double-click submissions is not the same as cleaning the entire customer database.

Build a **Customer Identity Reconciliation Tool** using:

- Normalized lowercase email
- Normalized phone number
- WordPress user ID
- Memberistic person ID
- Membership ID
- Formistic contact ID
- WooCommerce customer ID
- Stripe/Fortis customer ID
- Waiver archive ID

The tool should suggest merges but never automatically combine uncertain records.

Create exception groups:

- Exact email duplicates
  - Exact phone duplicates
  - Same name and date of birth
  - Multiple active memberships for one person
  - One membership connected to unrelated people
  - Family members connected to the wrong primary member
  - Waiver signed but not reflected on the account
  - Contact exists but no customer profile exists
- 

## 1.10 Memberships with incorrect linked people

Memberistic supports individual records, WordPress users, linked profiles, guest members, and corporate groups.

This flexibility also creates data-reconciliation risk.

Required membership audit rules:

- One person record per real person.
- One primary active membership per primary customer.
- Linked family profiles must reference the correct primary membership.
- A linked profile must not have a separate recurring charge unless intentionally purchased.
- Stripe/Fortis subscription ID must belong to the correct primary membership.
- Waivers remain individual, even when billing is shared.
- Cancelling a primary membership must apply the configured rule to linked profiles.
- Removing a person from a family must not delete their waiver history.

Generate a review screen where staff can detach, merge, or reassign linked profiles with a complete audit trail.

---

## 1.11 Replacing Stripe with an in-house-manageable processor

The client's concern is valid, but immediately removing Stripe would be dangerous.

The Booking Engine already supports several gateways, including Stripe, Fortis, PayPal, Authorize.Net, pay-in-store, and WooCommerce gateways.

Memberistic recurring billing is still primarily Stripe-specific.

### Recommended approach

**Do not shut Stripe down first.**

Build a Memberistic payment-gateway abstraction:

```
Membership Gateway Interface
├─ create_customer()
├─ create_subscription()
├─ cancel_subscription()
├─ update_payment_method()
├─ process_webhook()
├─ refund_payment()
├─ reconcile_subscription()
└─ create_customer_portal()
```

Then implement:

- Stripe adapter for existing subscriptions
- Fortis or approved merchant-service adapter for new subscriptions
- Unified payment ledger

- Gateway-independent cancellation UI
- Gateway-independent reconciliation
- Gateway-specific webhook logging

Migration sequence:

1. Keep existing Stripe subscriptions running.
  2. Put new subscriptions through the selected new gateway.
  3. Verify new signup, renewal, decline, cancellation, refund, and webhook flows.
  4. Move existing customers only when payment credentials can be migrated securely.
  5. When token migration is impossible, request payment reauthorization from members.
  6. Remove Stripe only after no active subscriptions remain.
- 

## 2. What you have already built for the business

You have created the foundation for six major business outcomes.

### 2.1 Range operations

- Online lane reservations
- Class registration
- Live availability
- Front-desk check-in
- Walk-ins
- Waiver verification
- Lane assignment
- Payment tracking
- Receipts
- Staff activity logs

### 2.2 Membership revenue

- Membership plans
- Recurring Stripe payments
- Linked members
- Corporate groups
- Digital membership cards
- QR verification
- Member discounts
- Renewal and cancellation workflows

### 2.3 Firearms retail and compliance

- FFL checkout
- Dealer selection

- Transfer lifecycle
- NICS workflow
- Bound-book support
- Age verification
- POS integration
- Lipsey's catalog and inventory architecture

## 2.4 Customer database

- WordPress users
- Membership people
- Waivers
- Bookings
- Orders
- Contact submissions
- Newsletter subscribers
- Communication records

## 2.5 Communication and retention

- Transactional emails
- Booking reminders
- SMS engine
- Contact inbox
- Newsletter system
- AI-assisted replies
- Chatbot replacement

## 2.6 Marketing growth

- SEO-focused theme
- Local-business structured data
- Landing pages
- Product and class pages
- Newsletter capture
- Customer remarketing foundation
- Business reporting dashboard

The technical foundation is substantial. The next objective is to make these systems behave as **one predictable business workflow**.

---

### 3. The biggest project risk: repository versus production

The July 8 incident report explicitly states that fixes in GitHub were not automatically deployed because releases were installed through manual ZIP uploads.

There is also visible documentation drift:

- The July 13 release report lists Memberistic 1.12.0, POS 3.2.0, Booking Engine 1.9.9.12, and Advanced FFL Checkout 1.15.0.
- `INSTALL.md` still lists several older versions, including Memberistic 1.10.7, POS 3.1.9, and Advanced FFL Checkout 1.9.4.

This means there is no fully reliable deployment manifest.

Before any more feature work, create a production inventory containing:

- Component
- Production version
- Repository version
- Release ZIP version
- Database schema version
- Configuration status
- Last deployment date
- Test result
- Rollback package

---

## 4. Master next-step plan

### Phase 0 — Establish one source of deployment truth

1. Back up production files and database.
2. Create a production-like staging clone.
3. Record every active theme and plugin version.
4. Compare production against the July 13 unified releases.
5. Correct `INSTALL.md` and create `PRODUCTION-MANIFEST.md`.
6. Stop making untracked direct changes on production.
7. Use a deployment checklist with rollback for every update.

## **Phase 1 — Resolve trust and revenue hotfixes**

Complete in this order:

1. Membership cancellation and remote subscription verification
2. Correct email address and business information
3. Class/calendar visibility
4. Booking payment status and webhook verification
5. Customer login/cache testing
6. Contact-form delivery verification
7. Chatbot operational verification

Every issue must have an acceptance test and evidence.

## **Phase 2 — Reconcile business data**

Build and run:

- Customer duplicate report
- Membership duplicate report
- Linked-member exception report
- Payment-subscription mismatch report
- Waiver-to-customer match report
- WordPress-user-to-member match report
- Contact and subscriber cleanup report

Do not delete records during the first pass. Mark candidates and review them.

## **Phase 3 — Complete Otter migrations**

1. Export Otter Waiver.
2. Mirror every available signed PDF.
3. Verify imported waiver counts.
4. Match waivers to accounts.
5. Test waiver status during check-in.
6. Verify the new chatbot and communication systems.
7. Disable Otter embeds and webhooks.
8. Cancel Otter services only after the acceptance gates pass.

## **Phase 4 — Launch the range counter system**

1. Configure staff roles.
2. Create the counter page and bookmark.
3. Confirm scanner/QR operation.
4. Test member, guest, walk-in, expired waiver, and unpaid scenarios.
5. Prepare the one-page checklist.
6. Train employees with test customers.

7. Require the manager to complete a full shift simulation.
8. Record final corrections.

## **Phase 5 — Stabilize Lipsey's and retail**

1. Separate firearms and accessories accounts.
2. Validate credentials and account identifiers.
3. Capture and inspect actual API payloads.
4. Complete category mapping.
5. Validate image retrieval.
6. Import into staging catalog.
7. Promote controlled test products to WooCommerce.
8. Test add-to-cart, FFL selection, payment, order, and transfer creation.
9. Enable scheduled synchronization only after the manual tests pass.

## **Phase 6 — Introduce the new merchant service safely**

1. Confirm the processor supports the required business and recurring billing model.
2. Implement the Memberistic gateway interface.
3. Add the new gateway adapter.
4. Test complete subscription lifecycle.
5. Put new memberships on the new gateway.
6. Reconcile Stripe and new-gateway subscriptions together.
7. Migrate or reauthorize existing members.
8. Retire Stripe only after its active-subscription count reaches zero.

## **Phase 7 — Website corrections and client handoff**

1. Inventory every public page.
2. Record each requested correction.
3. Correct and test desktop/mobile versions.
4. Verify forms, buttons, prices, hours, addresses, and links.
5. Create the restricted content-manager role.
6. Provide the editing guide.
7. Record the handoff walkthrough.
8. Obtain page-by-page acceptance.

## **Phase 8 — Email, SMS, and social growth system**

Create a clean marketing audience segmented into:

- Active members
- Expired members
- Range visitors
- Class attendees
- Retail customers
- FFL transfer customers



- Newsletter subscribers
- Leads who have not purchased

Then build campaigns for:

- Summer indoor-range promotion
- Membership conversion
- Class enrollment
- Ladies Day
- Retail offers
- Suppressor/NFA services
- Lapsed-member reactivation
- Review requests

Maintain:

- Consent source
- Subscription status
- SMS opt-out
- Email unsubscribe
- Last interaction
- Last purchase
- Membership status

Measure range traffic and retail sales separately. Strong range usage with weak retail sales means the marketing opportunity is to convert existing visitors into product, training, membership, and repeat-purchase customers.

---

## 5. Definition of “operational”

The system should only be presented as operational when all of these pass:

- A membership cancellation stops remote billing.
- Failed cancellations are visible and actionable.
- Every email contains correct business information.
- Staff can see and manage classes in the calendar.
- A customer can register, log in, reset a password, and access their account.
- A walk-in can complete the entire range check-in process.
- Waiver status is accurate and traceable.
- Otter waivers have been imported and backed up.
- Contacts and memberships have been checked for duplicates.
- Linked family profiles have been verified.
- Lipsey's firearms and accessories feeds operate independently.
- Product categories and images import correctly.
- Staff can use the system from a written checklist.
- The client can edit approved website content.

- Payment, booking, membership, email, SMS, and check-in actions have logs.
- Every core process has a rollback or manual fallback.

## Final strategic direction

The best next move is:

**Freeze feature expansion, synchronize production with the repository, fix revenue and trust issues, reconcile customer data, deliver the front-counter workflow, and complete staff/client handoff.**

After that, continue with merchant-service replacement, Lipsey's automation, and the email/SMS/social growth program.